# Week 01 - Descriptives

**Action:** Environment setup, load dataset, read GitSkills documentation closely **Outcome:** A working, queryable environment with basic descriptives for data

**Dataset:** GitSkills July 2026 Snapshot

- Research Paper DOI: 10.48550/arXiv.2608.10906
- Dataset DOI: 10.5281/zenodo.21875637

**Data Source:**

- Hugging Face Parquet Mirror: mvaccargiu/gitskills
- commit: ebab17454a7c236f8f26b183567f1a126f42e3f8

## 1. Database and Notebook Setup

In [1]:
```
from pathlib import Path
import duckdb
import pandas as pd
import matplotlib.pyplot as plt

ROOT = Path.cwd().parent          # notebooks/ -> project root
RESULTS = ROOT / "results"
FIGURES = ROOT / "figures"
RESULTS.mkdir(exist_ok=True)
FIGURES.mkdir(exist_ok=True)

con = duckdb.connect(str(ROOT / "data" / "gitskills.db"), read_only=True)
```

## 2. Data Loaded

Done to verify the same quantities of data are loaded as described here. Purposely leaving out artifact siblings as current work doesn't require that data, and it is a large portion of data that would slow research.

In [2]:
```
con.sql("""
    SELECT 'artifacts' AS table_name, count(*) AS n_rows FROM artifacts
    UNION ALL SELECT 'repos', count(*) FROM repos
    UNION ALL SELECT 'mining_runs', count(*) FROM mining_runs
""").df()
```

Out[2]:

| | table_name | n_rows |
|---|---|---|
| **0** | artifacts | 3797117 |
| **1** | repos | 282200 |
| **2** | mining_runs | 7 |

## 3. Overview of Tables

This is the overall layout of key tables within the GitSkills dataset. It includes column names and datatypes for each table.

In [3]:
```
con.sql("DESCRIBE repos").df()[["column_name", "column_type"]]
```

Out[3]:

| | column_name | column_type |
|---|---|---|
| 0 | full_name | VARCHAR |
| 1 | owner | VARCHAR |
| 2 | stars | BIGINT |
| 3 | forks | BIGINT |
| 4 | is_fork | BIGINT |
| 5 | language | VARCHAR |
| 6 | license | VARCHAR |
| 7 | description | VARCHAR |
| 8 | created_at | VARCHAR |
| 9 | pushed_at | VARCHAR |
| 10 | metadata_fetched | BIGINT |

In [4]:
```
con.sql("DESCRIBE artifacts").df()[["column_name", "column_type"]]
```

Out[4]:

| | column_name | column_type |
|---|---|---|
| 0 | repo_full_name | VARCHAR |
| 1 | path | VARCHAR |
| 2 | filename | VARCHAR |
| 3 | location_class | VARCHAR |
| 4 | file_sha | VARCHAR |
| 5 | discovered_at | VARCHAR |
| 6 | content | VARCHAR |
| 7 | content_fetched | BIGINT |
| 8 | frontmatter_valid | BIGINT |
| 9 | name | VARCHAR |
| 10 | description | VARCHAR |
| 11 | body_chars | BIGINT |
| 12 | history_fetched | BIGINT |
| 13 | composition_fetched | BIGINT |
| 14 | dedup_primary | BIGINT |
| 15 | first_commit_at | VARCHAR |
| 16 | last_commit_at | VARCHAR |
| 17 | commit_count | BIGINT |
| 18 | sibling_count | BIGINT |
| 19 | sibling_bytes | BIGINT |
| 20 | has_scripts | BIGINT |
| 21 | has_references | BIGINT |
| 22 | content_sha_ok | BIGINT |
| 23 | composition_truncated | BIGINT |
| 24 | first_commit_author | VARCHAR |
| 25 | first_commit_author_type | VARCHAR |
| 26 | first_commit_message | VARCHAR |
| 27 | last_commit_author | VARCHAR |
| 28 | last_commit_author_type | VARCHAR |
| 29 | last_commit_message | VARCHAR |

In [5]:
```
# Temporarily remove col width limits to display full text
with pd.option_context('display.max_colwidth', None):
    display(con.sql("SELECT * FROM mining_runs").df())
```

| | run_id | artifact_type | query | started_at | finished_at | discovered | note |
|---|---|---|---|---|---|---|---|
| **0** | 1 | agent-skills | filename:SKILL.md | 2026-07-09T17:26:45.768703+00:00 | 2026-07-09T18:04:34.751737+00:00 | 11796 | NaN |
| **1** | 2 | agent-skills | filename:SKILL.md | 2026-07-09T18:07:39.150068+00:00 | 2026-07-12T06:54:51.308396+00:00 | 1328473 | NaN |
| **2** | 3 | agent-skills | filename:SKILL.md | 2026-07-12T06:59:13.601456+00:00 | 2026-07-20T22:26:56.553671+00:00 | 0 | closed retroactively: process crashed before finishing (repair-runs) |
| **3** | 4 | agent-skills | filename:SKILL.md | 2026-07-13T12:52:52.309967+00:00 | 2026-07-20T22:26:56.553671+00:00 | 0 | closed retroactively: process crashed before finishing (repair-runs) |
| **4** | 5 | agent-skills | filename:SKILL.md | 2026-07-14T18:04:23.050325+00:00 | 2026-07-20T05:23:31.638481+00:00 | 1657948 | NaN |
| **5** | 6 | agent-skills | filename:SKILL.md | 2026-07-20T05:28:44.667482+00:00 | 2026-07-20T22:26:56.553671+00:00 | 0 | closed retroactively: process crashed before finishing (repair-runs) |
| **6** | 7 | agent-skills | filename:SKILL.md | 2026-07-20T07:38:28.581854+00:00 | 2026-07-20T22:26:56.553671+00:00 | 0 | closed retroactively: process crashed before finishing (repair-runs) |

## 4. Profiling Tables

This section analyzes the contents of the repo column currently. More profiling will be done in next weeks report.

In [6]:
```
cols = ["column_name", "column_type", "min", "max", "approx_unique", "q50", "null_percentage"]

repos_profile = con.sql("SUMMARIZE SELECT * EXCLUDE (description) FROM repos").df()
repos_profile.to_csv(RESULTS / "week01" / "repos_profile.csv", index=False)

repos_profile[cols].rename(columns={"q50": "median", "approx_unique": "unique_values"})
```

Out[6]:

| | column_name | column_type | min | max | unique_values | median | null_percentage |
|---|---|---|---|---|---|---|---|
| **0** | full_name | VARCHAR | 0-AI-UG/cate | zzzzz1213/Nano-Code-Agent | 232901 | NaN | 0.00 |
| **1** | owner | VARCHAR | 0-AI-UG | zzzzz1213 | 192218 | NaN | 0.00 |
| **2** | stars | BIGINT | 0 | 383040 | 3520 | 0 | 0.18 |
| **3** | forks | BIGINT | 0 | 80428 | 1652 | 0 | 0.18 |
| **4** | is_fork | BIGINT | 0 | 1 | 2 | 0 | 0.18 |
| **5** | language | VARCHAR | | xBase | 258 | NaN | 0.18 |
| **6** | license | VARCHAR | | Zlib | 44 | NaN | 0.18 |
| **7** | created_at | VARCHAR | 2008-01-27T17:23:23Z | 2026-07-21T02:24:50Z | 262796 | NaN | 0.18 |
| **8** | pushed_at | VARCHAR | 2015-05-08T03:22:57Z | 2026-07-21T08:49:38Z | 227038 | NaN | 0.18 |
| **9** | metadata_fetched | BIGINT | 1 | 1 | 1 | 1 | 0.00 |

# 5. Key Details

Some key details I have found, some verifying claims made by dataset creators. This is to support week one's basic descriptives goal.

## Copies

This covers how many duplications there are in the dataset. The results of 50.5% match what the dataset paper reports. The intial report is at the following link under "Intended Uses" for the GitSkills dataset

In [7]:
```
con.sql("""
    SELECT
        count(*) AS total_files,
        count(DISTINCT file_sha) AS distinct_contents,
        count(*) - count(DISTINCT file_sha) AS copies,
        round(100.0 * (total_files - distinct_contents) / total_files, 1) AS pct_copies
    FROM artifacts
""").df()
```

Out[7]:

| | total_files | distinct_contents | copies | pct_copies |
|---|---|---|---|---|
| **0** | 3797117 | 1877981 | 1919136 | 50.5 |

## Skill Locations

This breaks down where skill files are within projects by individual location.

In [8]:
```
con.sql("""
    SELECT location_class,
        count(*) AS num_files,
        round(100.0 * num_files / sum(num_files) OVER (), 1) AS pct
    FROM artifacts
    GROUP BY location_class
    ORDER BY num_files DESC
""").df()
```

Out[8]:

| | location_class | num_files | pct |
|---|---|---|---|
| **0** | skills-dir | 2102053 | 55.4 |
| **1** | other | 1321412 | 34.8 |
| **2** | canonical | 373652 | 9.8 |

### How Widely Are Skills Shared

This counts how many repositories each unique skill appears in, which relates to the core research of whether skills are project specific or widely shared.

In [9]:
```
con.sql("""
    WITH individual_skill AS (
        SELECT file_sha, count(DISTINCT repo_full_name) AS num_repos
        FROM artifacts
        GROUP BY file_sha
    )
    SELECT CASE
        WHEN num_repos = 1    THEN '1 repo'
        WHEN num_repos <= 10  THEN '2-10 repos'
        WHEN num_repos <= 100 THEN '11-100 repos'
        ELSE '100+ repos' END AS shared_across,
    count(*) AS num_skills,
    round(100.0 * num_skills / sum(num_skills) OVER (), 3) AS pct
    FROM individual_skill
    GROUP BY shared_across
    ORDER BY min(num_repos)
""").df()
```

Out[9]:

| | shared_across | num_skills | pct |
|---|---|---|---|
| **0** | 1 repo | 1571893 | 83.701 |
| **1** | 2-10 repos | 282613 | 15.049 |
| **2** | 11-100 repos | 23005 | 1.225 |
| **3** | 100+ repos | 470 | 0.025 |

## 6. Observations

**Loaded Data**

- I loaded all artifacts, repos, and mining_runs, and they all matched the published row counts. The data seems to have been collected in three runs between 2026-07-09 and 2026-07-20.

**Initial Findings For Broad vs. Project Specific Skills**

- Around half of the skill files (50.5%) are copies of another skill file
- Most of the unique skills are not shared, with 83.7% of the skills appearing in only one repository. 470 of the skills appear in more than 100 repositories.
- Copying seems to be concrentrated to a small set of widely shared skills, while most of the skills are written for individual projects.

**Skill Locations In Repositories**

- 9.8% of files are in the canonical location, or `.claude/skills`
- 55.4% of files live in other recognized skill directories
- 34.8% of files are elsewhere Analyzing skill location may help separate shared skills from project-specific skills.

**Repo Table Profiling**

- Most skills live in small projects, with the median stars and forks being zero
- Repository age shouldn't be used to determine skill creation date, as repo age goes back to 2008